React Native Mini Assignment: Property Listing & Booking App

1. Objective

Build a React Native CLI application that fetches real property data, displays property details, and allows users to submit a simulated property visit request.

Suggested time: 2-4 hours

2. Tech Stack

- React Native CLI with TypeScript preferred
- Functional components and React Hooks
- Fetch API or Axios
- React Navigation
- Shared theme and reusable components

3. APIs

- **Listings:** https://fakeapifordevs.vercel.app/api/realestate/listings?page=1&per_page=10
- **Property Details:** https://fakeapifordevs.vercel.app/api/realestate/listings/prop_001
- **Property Photos:** https://fakeapifordevs.vercel.app/api/realestate/listings/prop_001/photos
- **API Documentation:** https://fakeapifordevs.vercel.app/docs/realestate

Inspect the actual API responses and use the available fields and image URLs. Do not hardcode property cards or listing data. Fetch details and photos using the selected property ID.

4. Required Screens

Screen 1: Property Listing

Display property image, name/address, price, property type, bedrooms, bathrooms, and area where available. Include a button to view details.

Handle loading, API errors with retry, empty results, missing images, and optional fields.

Screen 2: Property Details

Fetch and display the selected property's photo gallery, full address, price, property type, bedrooms, bathrooms, area, description, and available amenities. Include a **Book a Visit** button and suitable loading and error states.

Screen 3: Book a Visit

Display a summary of the selected property and a form containing:

- Visitor name
- Email address
- Contact number
- Preferred visit date
- Preferred time slot

Validate required fields, email format, contact number, and past dates. Show submitting feedback and prevent duplicate submissions.

Screen 4: Booking Confirmation

After simulated submission, show a confirmation message with the selected property, visitor details, and preferred visit date and time.

Note: Booking is frontend-only. Do not claim that an actual appointment has been reserved.

5. UI Design, Fonts & Colors

Create a modern, minimal, premium real estate UI with a consistent **light theme**.

Google Fonts:

- **Fraunces:** Headings, property names, and prices
  https://fonts.google.com/specimen/Fraunces
- **Outfit:** Body text, descriptions, buttons, and forms
  https://fonts.google.com/specimen/Outfit

Download and integrate the font files into the React Native CLI project, and verify rendering on Android and iOS.

Color palette:

Element	Color
Background	Warm Off-White – #FAF9F6
Cards	White – #FFFFFF
Primary buttons	Forest Green – #264E36
Secondary accents	Sage Green – #A3B18A
Primary text	Charcoal – #252823
Secondary text	Muted Gray – #777B75
Borders	Light Gray – #E7E7E2
Accent	Soft Sand – #E9DFC8
Success	#E4F0E5
Error	#FCE8E6

Use consistent spacing, rounded cards and buttons, readable text contrast, subtle shadows, and responsive layouts across all screens.

6. Code Structure & Implementation

- Create reusable components such as PropertyCard, CustomButton, LoadingIndicator, and ErrorState.
- Separate API logic from UI components.
- Use React Hooks for API data, loading/error states, and form handling.
- Use TypeScript interfaces for property data where practical.
- Centralize theme values in files such as src/theme/colors.ts and src/theme/typography.ts.
- Handle network failures, missing fields, and invalid form inputs gracefully.
- Explain assumptions when the API does not supply a required field.

7. Deliverables

1. Working React Native application.
2. Source code in a Git repository or ZIP file.
3. README containing setup instructions, API endpoints, font integration, project structure, state management, booking flow, and assumptions.

8. Evaluation Criteria

API integration, navigation, React Native fundamentals, UI quality, font integration, state management, form validation, error handling, responsive design, code organization, and documentation.

Important: Prioritize a working application with correct API integration and reliable error handling. Be prepared to explain the code, debug an issue, or make a small requirement change during the review.